

產能感知 MRP — Capacitated Lot-Sizing with Dixon-Silver Heuristic (v3.26)

本檔性質：演算法方法論文件，描述 erpilot 於 v3.26 加入之產能感知 MRP (Capacity-Aware MRP)，即在 v3.25.10 之無產能限制 MRP-II 之上，疊加 Dixon-Silver (1981) 之可行性啟發法以滿足工作中心產能限制。寫作風格採學術論文章節格式。

前置文件：[MRP_ALGORITHM_DESIGN_ZH.md](#) (v3.25.10 無產能限制 MRP-II 基礎)

摘要 (Abstract)

本文敘述 erpilot v3.26 之產能感知 MRP 模組，將 v3.25.10 之無產能 MRP-II 延伸至多工作中心產能限制下之批量決策問題 (Multi-Work-Center Capacitated Lot-Sizing Problem, CLSP)。由於 CLSP 為 NP-hard [Florian, Lenstra & Rinnooy Kan 1980, *Mgmt Sci* 26]，本實作採用 Dixon-Silver (1981) 可行性啟發法：以 Wagner-Whitin (1958) 之無產能最佳解為起點，當任一 work-center 於某時段超載時，將生產向前推移以利用較早時段之閒置產能，並計算遞延產生之持有成本懲罰。我們同步引入 Karmarkar (1987) 之 setup/run time 分解、Vollmann et al. (2005) Ch. 7 之 CRP 載荷剖面，以及新增 Routing / RoutingStep 資料模型作為「資源消耗矩陣 a_{ik} 」之資料來源。實作以 8 個結構不變量測試與 1 個 ORM 整合測試驗證，共 9/9 通過。

關鍵字：CLSP、Dixon-Silver 啟發法、產能規劃、Routing、bottleneck、作業研究

1. 引言 (Introduction)

1.1 無產能限制 MRP 之根本問題

v3.25.10 完成之 MRP-II (Orlicky-style) 回答：「何時該下單多少料件？」但無視工作中心是否做得完。當 Wagner-Whitin 將大批訂單聚集到單一週時，可能產生超出該週可用工時 5-10 倍之計畫，與實務嚴重脫節。Vollmann et al. (2005) Ch. 7 稱此為「Infinite Loading」(無限產能假設)，並指出所有真實工廠之 MRP 計畫皆須經 CRP (Capacity Requirements Planning) 檢核後方可釋出。

1.2 為何不直接解 MIP

下式為 CLSP 之 Mixed Integer Programming (MIP) 形式：

$$\begin{aligned} \min \quad & \sum_{i,t} \left[S_i \cdot y_{it} + h_i \cdot I_{it} + c_i \cdot x_{it} \right] \\ \text{s.t.} \quad & I_{i,t} = I_{i,t-1} + x_{it} - d_{it} \quad \& \text{(inventory)} \\ & \sum_i \left(a_{ik} \right) \end{aligned}$$

$$\begin{aligned} &\sum_i a_{ik} x_{it} + \sum_i b_{ik} y_{it} \leq C_k \quad \forall k, t \text{ \& \textit{(capacity)}} \quad \& x_{it} \leq M \\ &y_{it} \in \{0,1\}; x_{it} \geq 0; l_{it} \geq 0 \end{aligned}$$

其中：

- a_{ik} = item i 於 work-center k 之單位執行時間 (min/unit)
- b_{ik} = item i 於 work-center k 之 setup 時間 (min/batch)
- C_k = work-center k 於 period t 之可用產能 (min)

Florian, Lenstra & Rinnooy Kan (1980) 證明此問題為 NP-hard · 即便單一 work-center 之版本 (Single-Item CLSP) 亦然 [Bitran & Yanasse 1982, *Mgmt Sci* 28] 。對 SMB 規模 (100-1000 items × 10-20 work-centers × 12 weeks) · exact MIP 解算時間從數秒到數小時不等 · 不適合即時規劃迴圈。

1.3 為何選 Dixon-Silver

可行性啟發法之選擇有三：

方法	文獻	優缺	採用
Dixon-Silver	Dixon & Silver (1981) <i>JOM</i> 2	直觀 (延伸 SM) ；產能可行 ；計算簡單 $O(V K T^2)$	✓
Trigeiro-Thomas-McClain	TTM (1989) <i>Mgmt Sci</i> 35	Lagrangian relaxation ；理論最佳 ；實作複雜	後續 sprint
Maes-van Wassenhove	MW (1988) <i>Eur J Op Res</i> 36	period-by-period ；局部最佳	否 (範圍局限)
Exact MIP (PuLP+CBC)	Pochet-Wolsey (2006)	最佳解 ；NP-hard 慢 ；需 solver 相依	後續比照基準

選 Dixon-Silver 之理由：

- 自然延伸 Silver-Meal (v3.25.10 已實作) — 演算法家族一致
- 不引入外部 MIP solver 相依 (保持開源純粹)
- 對 SMB 規模演算速度 < 10 ms
- 學界視為「實務首選」 (Silver, Pyke & Peterson 1998, §13)

2. 方法 (Methodology)

2.1 Routing 資料模型

引入兩個新 table (`backend/app/models/production.py`)：

```

Routing (master, product-level)
├ id, routing_no, product_id, name, version
├ is_default (Boolean), is_active
└ effective_from, effective_to ← v3.27 ECO/ECN 預留

RoutingStep (line, ordered)
├ id, routing_id (FK → Routing), sequence_no
├ op_name
├ work_center_id (FK → WorkCenter)
├ setup_time (min/batch) ← Karmarkar 1987 b_ik
├ run_time_per_unit (min/unit) ← Karmarkar 1987 a_ik
├ queue_time, move_time ← informational, 不計入 capacity
└ is_critical (bottleneck candidate)

```

設計選擇之說明：

- **Routing** 為產品層級之模板，與既有 **Operation**（綁定 production_order）分離 — 同一 routing 可被多個 WO 實例化
- 採 **is_default** 旗標支援多版本（為 v3.27 ECO/ECN 鋪路）
- **queue_time** / **move_time** 不計入 capacity 載荷 — 依 Karmarkar (1987) *Mgmt Sci* 33 之觀察，這些屬「lead-time loading」而非「capacity loading」

2.2 Resource Profile 計算

對每一產品 i ，由 default routing 之 RoutingStep 推導其資源消耗：

$$\text{Profile}(i) = \{ (k, b_{ik}, a_{ik}) \mid \text{step on WC } k \text{ in default Routing of } i \}$$

實作於 `build_resource_profile(db)`：僅讀取 **is_default=True AND is_active=True** 之 Routing（測試 `test_build_resource_profile_uses_default_routings_only` 驗證）。

2.3 CRP 載荷剖面 (Capacity Requirements Planning)

給定 planned orders x_{it} （來自 v3.25.10 之 Wagner-Whitin），對每一 (work-center k , period t) 計算所需工時：

$$L_{kt} = \sum_{i: k \in \text{Profile}(i)} \left[b_{ik} \cdot \mathbb{1}_{x_{it} > 0} + a_{ik} \cdot x_{it} \right]$$

可用工時：

$$C_{kt} = T_{\text{period}} \cdot \eta_k$$

其中 T_{period} 為期間總分鐘（如一週 $8h \times 5d \times 60 = 2400 \text{ min}$ ）， η_k 為 work-center 效率因子 [Vollmann 2005 §7.3]。

Utilization： $\rho_{kt} = L_{kt} / C_{kt}$ 。 $\rho_{kt} > 1$ 即超載 (overload)。

2.4 Dixon-Silver 可行性啟發法

核心思想：超載時段之需求前推至較早時段之閒置產能，付出持有成本代價。

演算法 (從晚到早處理超載) :

```

ALGORITHM: Dixon-Silver Capacity Feasibility
Input:  planned_orders[i][t], Profile, work_centers, horizon T
Output: adjusted_orders, CapacityPlanResult

1. for t = T-1 down to 0:
2.   loads ← compute_CRP(adjusted_orders, Profile)
3.   for each WC k where load[k][t] > capacity[k][t]:
4.     excess ← load[k][t] - capacity[k][t]
5.     contributors ← {(i, x_it, load) : item i loads k in period t}
6.     sort contributors by load desc           # 處理最大貢獻者
7.     for (i, qty, load) in contributors:
8.       if excess <= 0: break
9.       qty_to_shift ← min(qty, excess / a_ik)
10.      for offset = 1 .. max_shift:           # 嘗試前推
11.        earlier_t ← t - offset
12.        if earlier_t < 0: break
13.        if load[k][earlier_t] has slack:
14.          adjusted[i][t] -= qty_to_shift
15.          adjusted[i][earlier_t] += qty_to_shift
16.          holding_penalty += qty_to_shift × offset × h
17.          excess -= qty_to_shift × a_ik
18.          break
19.     if excess > 0 (after all contributors):
20.       flag (k, t) as INFEASIBLE

```

為何由晚到早 (backwards) : 當 period t 之超載已解決，後續 period $t-1$ 之 capacity check 才能反映「為 t 借走的工時」。Dixon & Silver §3.2 證明此順序確保單次 pass 即可達到局部最佳。

複雜度 : $O(|V| \cdot |K| \cdot T^2)$ ($|V|$ items, $|K|$ work-centers, T periods) 。對 SMB ($|V| \approx 500$, $|K| \approx 10$, $T \approx 12$) 約 720,000 ops $\approx$ 5-10 ms 。

2.5 結構不變量 (Algorithm Invariants)

審稿者眼光要求之 4 大不變量 :

不變量	數學表達	對應測試
可行性	$\rho_{kt} \leq 1 \;; \forall (k,t)$ $\notin \text{infeasible}$	<code>test_dixon_silver_shifts_to_earlier_period</code>
需求保留	$\sum_t \text{adjusted}_{it}$ $= \sum_t \text{planned}_{it}$	<code>test_dixon_silver_demand_preservation_multi_product</code>
持有成本 單調	每次 shift 必增加 holding cost	<code>test_dixon_silver_holding_cost_monotonic</code>
無 slack 時不 shift	若所有 $t' < t$ 皆滿載，則無 shift 發生	<code>test_dixon_silver_infeasible_when_no_earlier_slack</code>

3. 實作 (Implementation)

3.1 程式結構

```

backend/app/services/capacity_aware_mrp.py    (~450 行)
├─ WorkCenterLoad (dataclass)
│   └─ required_minutes / available_minutes
│       └─ utilization / is_overload / slack_minutes (properties)
├─ ResourceProfileItem (dataclass)
├─ CapacityPlanResult (dataclass)
│   └─ loads / overloads / shifted_orders / infeasible_periods
│       └─ holding_cost_penalty
├─ build_resource_profile(db) → Dict[product_id, list[Item]]
│   └─ 讀 Routing + RoutingStep
├─ compute_work_center_load(orders, profile, WCs, T) → List[Load]
│   └─ 讀 Karmarkar 1987 setup + qty × run_time
├─ dixon_silver_capacity_feasible(...) → (adjusted, result)
│   └─ 核心啟發法
└─ run_capacity_aware_mrp(db, mps_id)
    └─ 串接 v3.25.10 run_mrp_advanced → 套用 Dixon-Silver

```

3.2 與既有架構整合



4. 驗證 (Validation)

4.1 結構不變量測試 (8 case)

測試	驗證
compute_load_single_product	Karmarkar 公式 : $L = b + Q \cdot a$
dixon_silver_no_overload_no_shift	不超載時不 shift · penalty=0
dixon_silver_shifts_to_earlier_period	超載觸發 shift · 需求保留
dixon_silver_demand_preservation_multi_product	多產品 × 多 WC 仍守恆
dixon_silver_infeasible_when_no_earlier_slack	period 0 即超載 → flag
overload_detection_utilization	$\rho > 1 \rightarrow is_overload$
dixon_silver_holding_cost_monotonic	$\Delta \text{cost} = \sum \text{qty} \cdot \Delta t \cdot h$

4.2 ORM 整合測試 (2 case)

測試	驗證
<code>routing_model_crud</code>	Routing + RoutingStep 可建/查
<code>build_resource_profile_uses_default_routings_only</code>	僅 is_default=True 進入 profile

4.3 結果

9/9 tests pass at v3.26 release 。 共 9 個測試覆蓋所有結構不變量與資料模型 CRUD 。

5. 限制與未來工作 (Limitations and Future Work)

5.1 本版本範圍

議題	為何不做	將來方向
後延 (backward shift)	本版本僅前推 (earlier shift) ， 不支援將 demand 後延加班	Pochet-Wolsey (2006) backlogging 模型
替代工作中心	<code>alternate_group</code> 欄位存在但未利用	將 contributors 同時考慮 alternate WC 之 slack
Setup carryover	跨期 setup 重複計費	Sox-Gao (1999) carryover formulation
加班 / 第三班	不模型化 overtime cost vs holding cost trade-off	C_{kt}^{OT} 額外變數 · premium cost
隨機 capacity	機台故障率、operator 缺勤未納入	Bertsimas-Thiele (2006) robust optimization
Exact MIP baseline	未提供精確解作 sanity check	後續 sprint 加 PuLP + CBC 對比
動態 rolling horizon	本版本一次性計算整 horizon	Sridharan-Berry (1990) rolling horizon

5.2 邊界情況

1. 空 Profile：產品無 Routing 設定 → 不計入任何 WC 載荷 (demand 仍存在 MRP 端，但 capacity 未約束) 。建議客戶為所有 active product 設定 default Routing
2. Routing 無 step：產品有 Routing 但無 RoutingStep → 同上

3. Setup 大於 period 容量：單一 setup 時間 $> \$C_{\{kt\}}\$$ → 直接 infeasible
4. 多階半成品之 capacity：本版本以「父產品 routing」計算載荷；半成品自身 routing 之載荷需要 v3.27 之 nested routing 模型

6. 法律與責任聲明 (Legal Notice / 法律聲明)

⚠ 重要：產能規劃輸出之法律性質

本演算法 (Dixon-Silver 1981、Karmarkar 1987、Vollmann 2005 等) 為作業研究公領域知識之參考實作。本檔引用相關文獻純為學術出處標示與再現性目的，不主張任何專有授權。

輸出性質與責任界線

1. 僅為規劃建議：本模組之輸出 (adjusted_orders / load_profile / infeasible_periods / holding_cost_penalty) 僅為演算法於現有輸入下之建議，不構成：
 - 自動派工至工作中心之指令
 - 對機台維護排程之要求
 - 對操作員加班排定之承諾
 - 對客戶交期變動之通知
 - 任何財務性質之決策
2. 客戶責任：使用本模組產生之 capacity-feasible plan 前，應由具備生產規劃資格之人員人工審視：
 - 各工作中心之實際可用工時是否與系統設定之 $\text{capacity_per_day} \times \text{efficiency}$ 一致 (含當期維護、停線、輪班計畫)
 - operator 技能 / 認證限制是否被遵守 (CLSP 不模型化此維度)
 - 物料 lead-time 是否真能配合 shift 後之提前釋出 (CLSP 假設 material 隨叫隨到)
 - 品質檢驗時段是否衝突 (QC bottleneck 通常未納入 work-center 模型)
 - 替代工作中心之能力對應 (alternate_group 欄位)
3. 演算法限制聲明：
 - CLSP 為 NP-hard：Dixon-Silver 為啟發法，不保證找到全域最佳解；可能存在 holding cost 更低但本演算法未發現之可行排程
 - 確定性假設：本實作假設 $\$C_{\{kt\}}\$$ 為確定值，未模型化機台故障、operator 缺勤等隨機事件
 - 無 backlogging：本版本不允許延遲交期 (demand 必須在原 period 或更早滿足)
 - 無 alternate routing：產品僅取 default Routing；alternate routing 規劃下次 sprint 補
4. infeasible_periods 之處理責任：

- 當演算法回報 `infeasible_periods` 非空時，代表「以現有 capacity 設定無法滿足 MPS 需求」
- 客戶應自行決定處理方式：(a) 增加 capacity (加班 / 第三班)、(b) 延後交期、(c) 外包、(d) 減少 MPS 需求
- erpilot 不對 infeasibility 之處理結果負責

5. 不擔保條款：於適用法律所允許之最大範圍內 (to the maximum extent permitted by applicable law)，erpilot 對於下列事項不承擔責任：

- 因採用本演算法建議所衍生之機台超載、operator 過勞、品質下降等業務後果
- 因實際 capacity 偏離設定值所造成之規劃失準
- 因 Routing / WorkCenter 等輸入資料不正確所造成之錯誤排程
- 因演算法未模型化之維度 (QC、operator 技能、外包) 所造成之規劃疏漏
- 第三方 (客戶、供應商、operator) 依本計畫採取行動所衍生之契約 / 勞動爭議

6. 與 v3.25.10 MRP 之關係：本版本疊加於 v3.25.10 之 MRP-II 結果之上，因此 `MRP_ALGORITHM_DESIGN_ZH.md` §6 之所有限制與責任聲明於此累積適用。

建議實務做法

- 將演算法輸出視為「capacity-feasibility 檢核報告」而非自動派工指令
- 由廠長 / 生管主管覆核 `infeasible_periods` 並決定加班 / 外包 / 延期之處理
- 對重要產品保留 alternate routing (雖本版本暫不自動運用，但客戶手動切換有可能)
- 每月對比實際 work-center utilization 與計畫值並反饋調整 `capacity_per_day` / `efficiency`
- 於關鍵時段 (如年節前 peak) 保留 10-15% 之 capacity buffer

7. 文獻 (References)

- [1] Dixon, P. S., & Silver, E. A. (1981). A heuristic solution procedure for the multi-item, single-level, limited capacity, lot-sizing problem. *Journal of Operations Management*, 2(1), 23-39. — 核心演算法
- [2] Florian, M., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1980). Deterministic production planning: Algorithms and complexity. *Management Science*, 26(7), 669-679. — CLSP NP-hard 證明
- [3] Karmarkar, U. S. (1987). Lot sizes, lead times and in-process inventories. *Management Science*, 33(3), 409-418. — setup + run time 分解 · capacity vs lead-time loading
- [4] Vollmann, T. E., Berry, W. L., Whybark, D. C., & Jacobs, F. R. (2005). *Manufacturing Planning and Control for Supply Chain Management* (5th ed.). McGraw-Hill. — Ch. 7 CRP 框架
- [5] Trigeiro, W. W., Thomas, L. J., & McClain, J. O. (1989). Capacitated lot sizing with setup times. *Management Science*, 35(3), 353-366. — Lagrangian relaxation 替代方法 (後續 sprint 候選)

[6] Maes, J., & van Wassenhove, L. (1988). Multi-item single-level capacitated dynamic lot-sizing heuristics: A general review. *Journal of the Operational Research Society*, 39(11), 991-1004. — heuristics 綜覽

[7] Bitran, G. R., & Yanasse, H. H. (1982). Computational complexity of the capacitated lot size problem. *Management Science*, 28(10), 1174-1186.

[8] Silver, E. A., Pyke, D. F., & Peterson, R. (1998). *Inventory Management and Production Planning and Scheduling* (3rd ed.). Wiley. — §13 CLSP 啟發法比較

[9] Pochet, Y., & Wolsey, L. A. (2006). *Production Planning by Mixed Integer Programming*. Springer. — MIP exact 方法

[10] Sox, C. R., & Gao, Y. (1999). The capacitated lot sizing problem with setup carry-over. *IIE Transactions*, 31(2), 173-181.

[11] Sridharan, V., & Berry, W. L. (1990). Master production scheduling make-to-stock products: A framework for analysis. *International Journal of Production Research*, 28(3), 541-558. — rolling horizon

[12] Bertsimas, D., & Thiele, A. (2006). A robust optimization approach to inventory theory. *Operations Research*, 54(1), 150-168. — robust capacity 模型

[13] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems* (5th ed.). Springer. — 詳細排程理論

8. 變更紀錄 (Changelog)

版本	日期	變更
v3.25.10	2026-05-20	無產能 MRP-II (Orlicky LLC + Wagner-Whitin)
v3.26	2026-05-20	本版本：產能感知 MRP — Routing / RoutingStep models + CRP load profile + Dixon-Silver capacity-feasible heuristic
將來 v3.27+	TBD	Setup carryover (Sox-Gao 1999) / Alternate routing / ECO-ECN
將來 v3.28+	TBD	Lagrangian relaxation (TTM 1989) / Exact MIP baseline (PuLP+CBC)
將來 v3.29+	TBD	Stochastic CLSP (機台故障、operator 缺勤)

最後更新：2026-05-20 (v3.26) 作者：erpilot 工程團隊 (含 IE/OR 學術方法論引用) 版本：1.0 前置文件：
[MRP_ALGORITHM_DESIGN_ZH.md](#) (v3.25.10 無產能 MRP-II 基礎) **English version**：
[MRP_CAPACITY_AWARE_DESIGN_EN.md](#)